ECEC 661 – Homework 4

CORDIC Square Root IP (AXI-Stream core + custom AXI-Lite IP) on Cora Z7-07S

Gary Pham (gp492)

May 8, 2026

1 Problem Statement

Implement a Zynq PS+PL system that computes $\sqrt{x}$ using the Xilinx **CORDIC** IP configured for the **Square Root** function. Input x is presented as an unsigned **2Q7** fraction on the lower ten bits of a 16-bit word; output $z = \sqrt{x}$ is an unsigned **1Q8** fraction on the lower ten bits. Deliverables include HDL simulation of `user_logic`, a packaged custom IP that maps software-visible `slv_reg*` registers to the core, a Vivado block design, and a bare-metal C test application with UART evidence.

2 CORDIC IP Configuration

The Vivado IP Catalog instance `cordic_0` (`xilinx.com:ip:cordic:6.0`) was configured to match the assignment (Square Root, parallel architecture, maximum pipelining, unsigned fractional data, 10-bit in/out, truncate rounding):

Table 1: CORDIC IP settings used for `cordic_0`.

Parameter	**Value**
Functional Selection	Square Root
Architectural Configuration	Parallel
Pipelining Mode	Maximum
Data Format	UnsignedFraction
Phase Format	Radians
Input Width / Output Width	10 / 10
Round Mode	Truncate

The PG105 product guide describes how square-root mode interprets the Cartesian slave stream as the radicand and produces $\sqrt{x}$ on the master stream.

3 Block Design

Figure 1 shows the Vivado block diagram: Zynq PS (UART0 on MIO 14/15), `Processor System Reset`, an AXI interconnect or SmartConnect bridging `M_AXI_GP0` to the custom slave `cordic_sqrt_axi_0`, and clock/reset distribution at `FCLK_CLK0` (100 MHz typical). The custom IP was assigned base address `0x43C2_0000` (64 KB aperture), consistent with `sw/main.c` and `xparameters.h`.

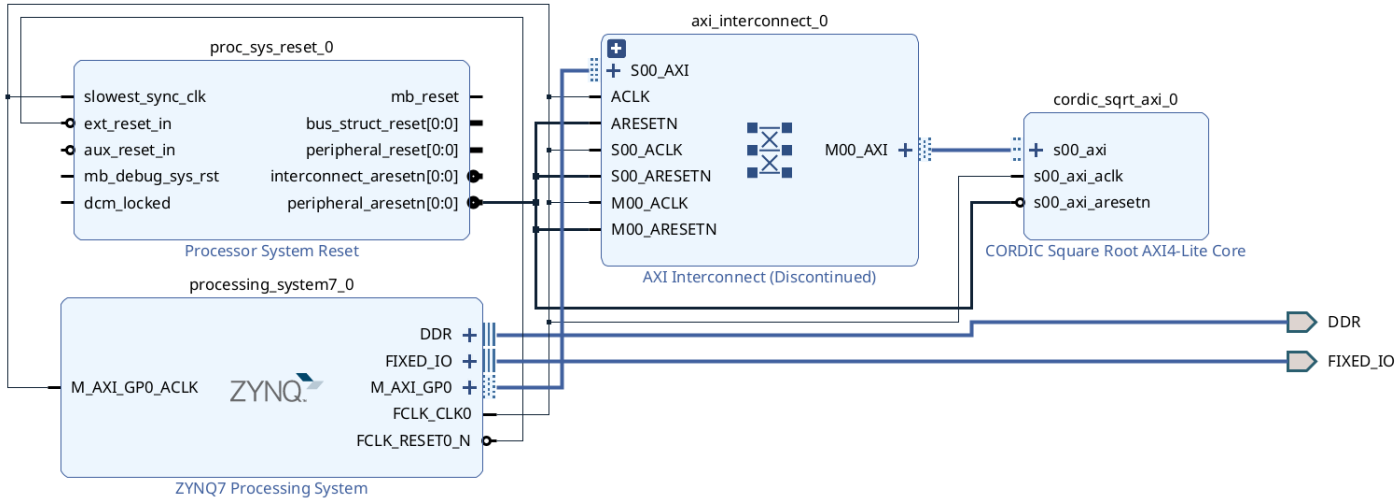

Figure 1: Vivado block design: Zynq PS, interconnect, and `cordic_sqrt_axi` AXI4-Lite peripheral.

4 User Logic and Custom IP

4.1 Assignment wrapper (`user_logic.vhd`)

`user_logic` preserves the required entity ports (`ck`, `aresetn`, `din_tvalid`, `x[15:0]`, `dout_tvalid`, `z[15:0]`) and instantiates `cordic_0`. The Vivado CORDIC exposes 16-bit AXI-Stream `tdata` lanes even for a 10-bit configuration (byte alignment); bits [15:10] are zero-stuffed on input and ignored on output. Bits [9:0] carry the 2Q7 radicand and 1Q8 result respectively:

```
s_tdata(9 downto 0)   <= x(9 downto 0);
s_tdata(15 downto 10) <= (others => '0');

U_CORDIC : cordic_0
    port map (
        aclk                    => ck,
        s_axis_cartesian_tvalid => din_tvalid,
        s_axis_cartesian_tdata  => s_tdata,
        m_axis_dout_tvalid      => dout_tvalid,
        m_axis_dout_tdata       => m_tdata
    );

z(9 downto 0)   <= m_tdata(9 downto 0);
z(15 downto 10) <= (others => '0');
```

Listing 1: Core mapping in `Homework_4/cordic_sqrt/rtl/user_logic.vhd` (excerpt).

4.2 AXI4-Lite slave (`cordic_sqrt_axi_v1_0_S00_axi.vhd`)

The generated-style slave implements four 32-bit registers. Writes program x and `din_tvalid`; reads return z and `dout_tvalid`. This matches the assignment reference snippet (`mWriteReg`/`mReadReg` offsets 0, 4, 8, 12):

Table 2: Software register map (byte offsets from IP base).

Offset	Name	R/W	Purpose
0x00	`slv_reg0`	R/W	x: lower 10 bits = 2Q7
0x04	`slv_reg1`	R/W	bit 0 = `din_tvalid`
0x08	`slv_reg2`	R	z: lower 10 bits = 1Q8
0x0C	`slv_reg3`	R	bit 0 = `dout_tvalid`

Inside the slave, `core_x <= slv_reg0(15 downto 0)`, `din_tvalid <= slv_reg1(0)`, and the read multiplexer drives `slv_reg2`/`slv_reg3` paths from `core_z` and `core_dout_tv`. Unlike Homework 3's accumulator, there is no `slv_reg_wren`-as-clock trick: the CORDIC runs continuously on `S_AXI_ACLK`.

5 Behavioral Simulation

`tb/tb_user_logic.vhd` exercises reset release, four consecutive samples with `din_tvalid` high (`0x080`, `0x040`, `0x008`, `0x000`), then compares pipelined outputs when `dout_tvalid` is asserted. Expected values match the specification (`0x100`, `0x0B5`, `0x040`, `0x000`).

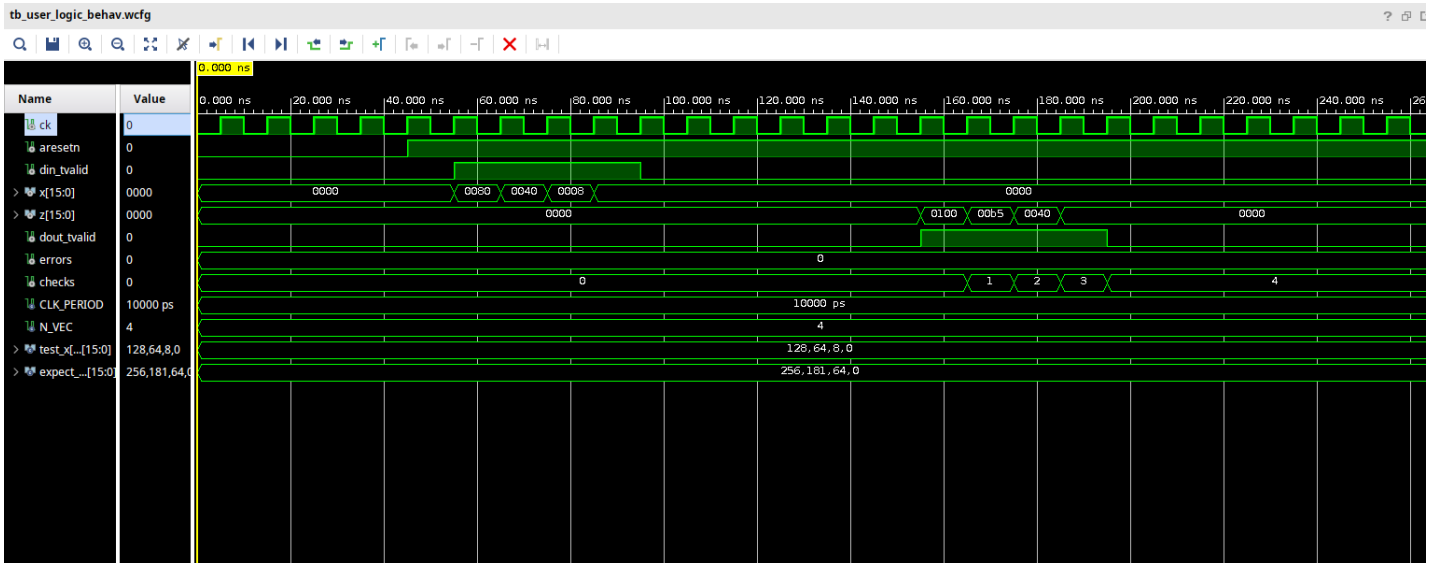

Figure 2: Vivado xsim waveform: `ck`, `aresetn`, stream handshake, `x`, `dout_tvalid`, and `z`.

6 Software (`main.c`)

The bare-metal application (`Homework_4/cordic_sqrt/sw/main.c`) follows the assignment handshake exactly: write x, assert `din_tvalid`, spin on `dout_tvalid`, deassert `din_tvalid`, then read z. Fixed-point decimals for the UART table are formatted without floating-point (`snprintf` with scaled milli-fracts), and column widths align header, rule line, and rows.

```c
for (int i = 0; i < N_TESTS; i++)
{
    CORDIC_SQRT_mWriteReg(BASEADDR, 0, test_vector[i]);  /* slv_reg0 = x */
    CORDIC_SQRT_mWriteReg(BASEADDR, 4, 1);               /* din_tvalid    */

    while (!CORDIC_SQRT_mReadReg(BASEADDR, 12))          /* dout_tvalid   */
        ;

    CORDIC_SQRT_mWriteReg(BASEADDR, 4, 0);               /* lower din_tv  */

    u32 x_in = CORDIC_SQRT_mReadReg(BASEADDR, 0);
    u32 z_out = CORDIC_SQRT_mReadReg(BASEADDR, 8);

    print_table_row(i, x_in, z_out, chk);
    ...
}
```

Listing 2: Register handshake loop (core of `main.c`).

Q-format display. With x_{10} and z_{10} the 10-bit lanes, $\text{value}_x = x_{10}/128$ (2Q7) and $\text{value}_z = z_{10}/256$ (1Q8). The printed table labels columns `x dec 2Q7` and `z dec 1Q8` accordingly.

Test vectors. Required cases 1, 1/2, 1/16 appear together with extra checks (0, 1/4, 0.125, 3) to stress truncation and dynamic range.

7 Hardware Results

Figure 3 shows UART output from the Vitis run on the Cora board: seven rows, each `OK`, and a passing summary. This satisfies the PuTTY / serial-monitor deliverable.

```
OUTPUT ×   /DEV/TTYUSB1 (DIGILENT) ×   DEBUG CONSOLE ×   TASK: XSDB CONSOLE   PROBLEMS 1 ×

cordic_sqrt starting in 3...
cordic_sqrt starting in 2...
cordic_sqrt starting in 1...

============================================================
   ECEC 661 HW4 - CORDIC Square Root IP test app
   IP base address : 0x43C20000
============================================================

cordic sqrt

  (x in 2Q7: value = x/128  |  z in 1Q8: value = z/256)

  # |  x(hex) |   x dec 2Q7 |  z(hex) |   z dec 1Q8 |  chk
--- | ------- | ----------- | ------- | ----------- | ----
  0 |   0x000 |  0.000      |   0x000 |  0.000      |   OK
  1 |   0x080 |  1.000      |   0x100 |  1.000      |   OK
  2 |   0x040 |  0.500      |   0x0B5 |  0.707      |   OK
  3 |   0x008 |  0.063      |   0x040 |  0.250      |   OK
  4 |   0x020 |  0.250      |   0x080 |  0.500      |   OK
  5 |   0x010 |  0.125      |   0x05A |  0.352      |   OK
  6 |   0x180 |  3.000      |   0x1BB |  1.730      |   OK

------------------------------------------------------------
   Summary : 7 passed, 0 failed (total 7)
------------------------------------------------------------
```

Figure 3: UART trace from the bare-metal CORDIC square-root test (formatted table + summary).

Representative expected hex pairs for the three graded vectors:

Table 3: Assignment reference vectors (2Q7 input $\rightarrow$ 1Q8 $\sqrt{x}$).

x (hex)	x decimal (2Q7)	z (hex)	z decimal (1Q8)
`0x080`	1.000	`0x100`	1.000
`0x040`	0.500	`0x0B5`	≈ 0.707
`0x008`	0.0625	`0x040`	0.250

8 Conclusion

The design wraps Xilinx `cordic_0` in assignment-compliant `user_logic`, packages it behind an AXI4-Lite slave with the reference register map, verifies the three prescribed fractions (plus extra cases) in simulation and on silicon, and prints a readable 2Q7/1Q8 table over UART for grading evidence.